

Введение в хранилища данных для аналитики

Про спикера

- ✓ Сертифицированный консультант в сфере Big Data и опытом в IT более 15 лет
- ✓ Опыт работы с проектами хранилищ с петабайтами данных
- ✓ Стриминговая обработка 500k событий в секунду
- ✓ Сферы: авто, финансовые технологии, проект Arche

Цель модуля

Узнать о различных типах хранилищ данных и об их применении в аналитических решениях.

Зачем?

Чтобы понять преимущества и недостатки и лучше понимать архитектуру проектов.
Общаться свободно на тех. интервью.

Задачи модуля

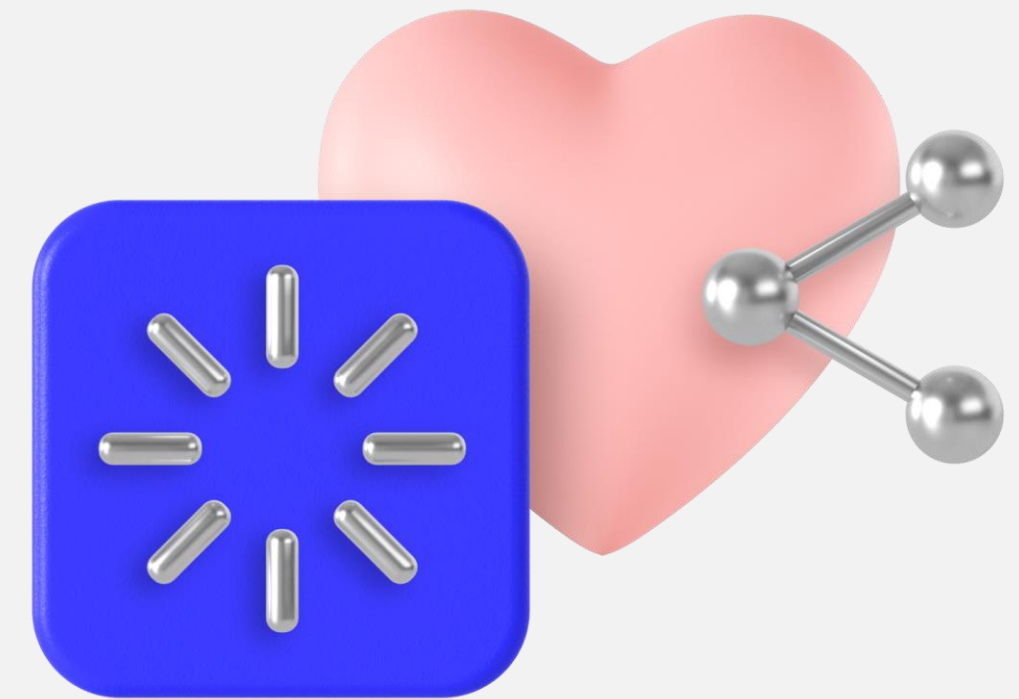
- 1 Познакомиться с типами баз данных
- 2 Понять отличия между OLAP и OLTP
- 3 Познакомиться с объектными хранилищами
- 4 Рассмотреть паттерны: DWH, Data Lake, Lake House

Типы баз данных

Цели

- ✓ Познакомиться с разными типами баз данных
- ✓ Понять, для каких задач использовать

Повтор или изучение некоторых понятий



Повторение — мать учения

1 DML

2 DDL

3 Таблицы

4 Индексы

5 Джойны

6 Ключи

7 Партиции

Data Manipulation Language (DML)

SELECT извлекает данные из одной или нескольких таблиц на основе указанных условий.

Пример: `SELECT * FROM employees WHERE department = 'Sales';`

INSERT вставляет новые данные в таблицу.

Пример: `INSERT INTO customers (name, email) VALUES ('John Doe', 'john@example.com');`

UPDATE изменяет существующие данные в таблице.

Пример: `UPDATE employees SET salary = 50000 WHERE department = 'Sales';`

DELETE удаляет данные из таблицы.

Пример: `DELETE FROM customers WHERE id = 123;`

Data Definition Language (DDL)

CREATE создаёт новый объект базы данных, такой как таблица, индекс или представление.

Пример: `CREATE TABLE employees (id INT, name VARCHAR(50), salary DECIMAL(10,2));`

ALTER модифицирует существующий объект базы данных, позволяя добавлять, изменять или удалять колонки, ограничения и другие атрибуты.

Пример: `ALTER TABLE employees ADD COLUMN age INT;`

DROP удаляет существующий объект базы данных, такой как таблица или индекс.

Пример: `DROP TABLE employees;`

TRUNCATE удаляет все данные из таблицы, но оставляет саму таблицу.

Пример: `TRUNCATE TABLE employees;`

Таблицы базы данных

Таблицы баз данных представляют структурированный способ хранения и организации данных, например, таблица «Студенты» с колонками «Имя», «Возраст» и «Оценка», где каждая строка представляет отдельного студента с их соответствующими данными.

```
CREATE TABLE Students (  
    Name VARCHAR(50),  
    Age INT,  
    Grade DECIMAL(3, 2)  
);
```

Индексы базы данных

Индекс баз данных — это структура данных, создаваемая для ускорения поиска, сортировки и извлечения данных из таблицы базы данных, позволяющая давать быстрый доступ к определённым значениям или диапазонам значений в определённых столбцах таблицы.

```
CREATE TABLE Students (  
    ID      INT,  
    Name    VARCHAR(50),  
    Age     INT,  
    Grade   DECIMAL(3, 2),  
    INDEX idx_Name (Name)  
);
```

Join в базе данных

Join в базе данных — это операция, которая объединяет данные из двух или более таблиц на основе совпадающих значений в определённых столбцах, позволяя комбинировать информацию из разных таблиц в один результат.

SELECT

```
Students.Name,  
Courses.CourseName
```

FROM Students

```
JOIN Courses ON Students.ID = Courses.StudentID;
```


Ключи в базе данных

Ключи в базе данных — это структуры данных, используемые для уникальной идентификации записей в таблице или для установления связей между таблицами, обеспечивая целостность данных и возможность эффективного поиска и сортировки.

```
CREATE TABLE Students (  
    ID          INT PRIMARY KEY,  
    Name       VARCHAR(50),  
    Age        INT,  
    Grade      DECIMAL(3, 2),  
    CourseID INT,  
    FOREIGN KEY (CourseID) REFERENCES Courses(ID)  
);
```

Партиции в базе данных

Партиции в базе данных — это метод разделения и организации больших таблиц на более мелкие и управляемые фрагменты для повышения производительности, улучшения обработки запросов и облегчения администрирования данных.

```
CREATE TABLE Sales (  
    SaleID INT,  
    Product VARCHAR(50),  
    Amount DECIMAL(10, 2),  
    Date DATE  
) PARTITION BY RANGE (DATE) (  
    PARTITION p1 VALUES LESS THAN ('2022-01-01'),  
    PARTITION p2 VALUES LESS THAN ('2023-01-01'),  
    PARTITION p3 VALUES LESS THAN MAXVALUE );
```

Типы базы данных

Relational

Document Oriented

Graph Oriented

Key Value

Реляционные базы данных

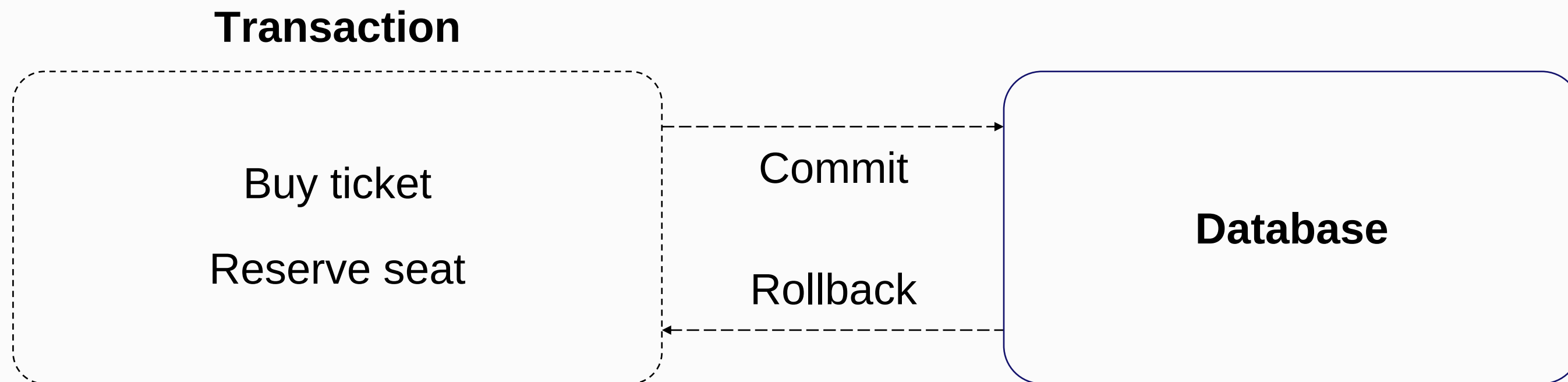
- 1 Microsoft SQL Server
- 2 Oracle Database
- 3 PostgreSQL
- 4 Amazon Aurora
- 5 MySQL
- 6 MariaDB

Реляционные базы данных

	Relational	Document Oriented	Key Value	Graph Oriented
DML	Full support			
DDL	Full support			
Tables Structure	Structured			
Indexes	Cluster & Non Cluster			
Join	Full support			
Keys	Primary & Foreign			
Partitions	Full support			

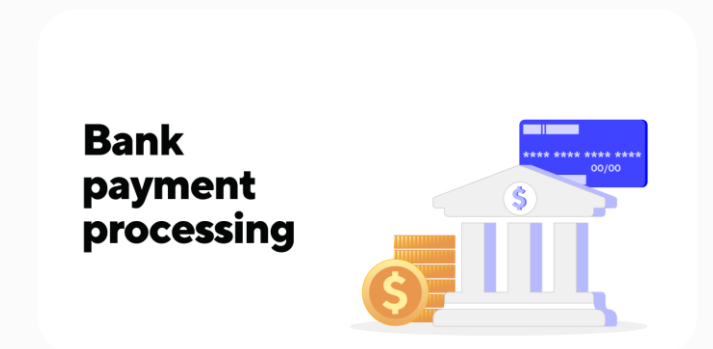
Транзакции

Логическая единица работы с базой данных, которая представляет собой последовательность одного или нескольких действий (операций), выполняемых в базе данных.



Когда применять?

- 1 Есть модель данных: структурированные сущности плюс связи
- 2 Много простых запросов на чтение
- 3 Много простых запросов на обновление и удаление
- 4 Нужна поддержка транзакций



Документоориентированные

- 1 MongoDB
- 2 Apache CouchDB
- 3 Amazon DocumentDB
- 4 Microsoft Azure
- 5 Cloud Firestore
- 6 YDB

Документоориентированные — структура

```
db.createCollection("Employee");
```

```
db.Employee.insertMany([
  {
    first_name: "John",
    last_name: "Doe",
    age: 30,
    department: "HR",
    position: "Manager",
    salary: 60000.00
  },
  {
    first_name: "Jane",
    last_name: "Smith",
    age: 28,
    department: "IT",
    position: "Developer",
    salary: 55000.00
  }
]);
```

Документоориентированные

	Relational	Document Oriented	Key Value	Graph Oriented
DML	Full support	Full support		
DDL	Full support	Full support		
Tables Structure	Structured	Collection		
Indexes	Cluster & Non Cluster	Full support		
Join	Full support	No (Denormalization)		
Keys	Primary & Foreign	Primary, Index keys		
Partitions	Full support	NO		

Когда применять?

- 1 Быстрое прототипирование приложений
- 2 Когда модель ещё не сформировалась
- 3 Когда сущность может содержать разное количество атрибутов
- 4 IoT — интернет вещей
- 5 Логгирование

Key Value

1 Cassandra

2 Tarantool

3 Apache Ignite

4 Redis

5 Amazon DynamoDB

6 Oracle NoSQL Database

7 Couchbase

Key Value

	Relational	Document Oriented	Key Value	Graph Oriented
DML	Full support	Full support	Limited support	
DDL	Full support	Full support	Full support	
Tables Structure	Structured	Collection	Structured	
Indexes	Cluster & Non Cluster	Full support	Limited	
Join	Full support	No (Denormalization)	No (Denormalization)	
Keys	Primary & Foreign	Primary, Index keys	Partition, Clustering	
Partitions	Full support	NO	Full support	

Когда применять?

- 1 Критичны низкие задержки
- 2 Быстрый поиск по ключу
- 3 Организация кэша
- 4 Хранилище меты
- 5 Онлайн-аналитика

Graph oriented

- 1 Neo4j
- 2 OrientDB
- 3 Amazon Neptune
- 4 Gremlin

Когда применять?

- 1 Социальные сети
- 2 Географические объекты
- 3 Рекомендательные системы
- 4 Антифрод

Пример добавления вершин и рёбер

```
g.addV('person').property('firstName', 'Thomas').property('lastName', 'Andersen')
    .property('age', 44).property('userid', 1).property('pk', 'pk')
g.addV('person').property('firstName', 'Mary Kay').property('lastName', 'Andersen')
    .property('age', 39).property('userid', 2).property('pk', 'pk')

g.V().hasLabel('person').has('firstName', 'Thomas')
    .addE('knows')
    .to(g.V().hasLabel('person').has('firstName', 'Mary Kay'))
```

Итоги

- 1 Рассмотрели основные типы БД и их основные характеристики
- 2 Разобрали случаи применения